

Minimizing the Accumulated Trajectory Error to Improve Dataset Distillation

Jiawei Du^{1,5,2†}, Yidi Jiang^{2†}, Vincent Y. F. Tan^{3,2}, Joey Tianyi Zhou^{1,5*}, Haizhou Li^{4,2}

¹Centre for Frontier AI Research (CFAR), Agency for Science, Technology and Research (A*STAR), Singapore

²Department of Electrical and Computer Engineering, National University of Singapore

³Department of Mathematics, National University of Singapore

⁴SRIBD, School of Data Science, The Chinese University of Hong Kong, Shenzhen, China

⁵Institute of High Performance Computing (IHPC), Agency for Science, Technology and Research (A*STAR), Singapore

{dujiawei, yidi-jiang}@u.nus.edu, vtan@nus.edu.sg, Joey.tianyi.zhou@gmail.com

Abstract

Model-based deep learning has achieved astounding successes due in part to the availability of large-scale real-world data. However, processing such massive amounts of data comes at a considerable cost in terms of computations, storage, training and the search for good neural architectures. Dataset distillation has thus recently come to the fore. This paradigm involves distilling information from large real-world datasets into tiny and compact synthetic datasets such that processing the latter ideally yields similar performances as the former. State-of-the-art methods primarily rely on learning the synthetic dataset by matching the gradients obtained during training between the real and synthetic data. However, these gradient-matching methods suffer from the so-called accumulated trajectory error caused by the discrepancy between the distillation and subsequent evaluation. To mitigate the adverse impact of this accumulated trajectory error, we propose a novel approach that encourages the optimization algorithm to seek a flat trajectory. We show that the weights trained on synthetic data are robust against the accumulated errors perturbations with the regularization towards the flat trajectory. Our method, called **Flat Trajectory Distillation (FTD)**, is shown to boost the performance of gradient-matching methods by up to 4.7% on a subset of images of the ImageNet dataset with higher resolution images. We also validate the effectiveness and generalizability of our method with datasets of different resolutions and demonstrate its applicability to neural architecture search. Code is available at <https://github.com/AngusDujw/FTD-distillation>.

1. Introduction

Modern deep learning has achieved astounding successes in achieving ever better performances in a wide range of

*Corresponding Author. † Equal Contribution.

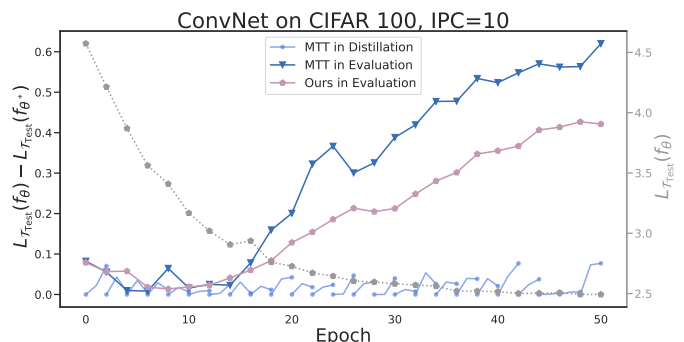


Figure 1. The change of the loss difference $L_{T_{Test}}(f_{\theta}) - L_{T_{Test}}(f_{\theta^*})$, in which θ and θ^* denote the weights optimized by synthetic dataset $\mathcal{S}$ and real dataset $\mathcal{T}$, respectively. The gray line represents $L_{T_{Test}}(f_{\theta^*})$ and is associated with the gray y-axis of the plot with two y-axes. The lines indicated by “Evaluation” represent the networks that are initialized at epoch 0 and trained with the synthetic dataset for 50 epochs. The line indicated by “Distillation” represents the network that is initialized at epochs 2, 4, ..., 48 and trained with the synthetic dataset for 2 epochs. The former lines have much higher loss difference compared to the latter; this is caused by the accumulated trajectory error. And we try to minimize it in the evaluation phase, so that the loss difference line of our method is lower and tends to converge than that of MTT [1].

fields by exploiting large-scale real-world data and well-constructed Deep Neural Networks (DNN) [4, 5, 9]. Unfortunately, these achievements have come at a prohibitively high cost in terms of computation, particularly when it relates to the tasks of data storage, network training, hyperparameter tuning, and architectural search.

A series of model distillation studies [2, 14, 16, 22] has thus been proposed to condense the scale of models by distilling the knowledge from a large-scale teacher model into a compact student one. Recently, a similar but distinct task, *dataset distillation* [1, 3, 26, 35, 36, 42–45, 47, 49, 50] has been considered to condense the size of real-world datasets. This task aims to synthesize a large-scale real-world dataset into a tiny synthetic one, such that a model trained with the synthetic dataset is comparable to the one trained with the

real dataset. Dataset distillation can expedite model training and reduce cost. It plays an important role in some machine learning tasks such as continual learning [39, 48–50], neural architecture search [19, 37, 38, 48, 50], and privacy-preserving tasks [8, 13, 27], etc.

Wang et al. [45] was the first to formally study dataset distillation. The authors proposed a method **DD** that models a regular optimizer as the function that treats the synthetic dataset as the inputs, and uses an additional optimizer to update the synthetic dataset pixel-by-pixel. Although the performance of **DD** degrades significantly compared to training on the real dataset, [45] revealed a promising solution for condensing datasets. In contrast to conventional methods, they introduced an evaluation standard for synthetic datasets that uses the learned distilled set to train randomly initialized neural networks and the authors evaluate their performance on the real test set. Following that, Such et al. [41] employed a generative model to generate the synthetic dataset. Nguyen et al. [34] reformulated the inner regular optimization of **DD** into a kernel-ridge regression problem, which admits closed-form solution.

In particular, Zhao and Bilen [50] pioneered a gradient-matching approach **DC**, which learns the synthetic dataset by minimizing the distance between two segments of gradients calculated from the real dataset $\mathcal{T}$, and the synthetic dataset $\mathcal{S}$. Instead of learning a synthetic dataset through a bi-level optimization as **DD** does, **DC** [50] optimizes the synthetic dataset explicitly and yields much better performance compared to **DD**. Along the lines of **DC** [50], more gradient-matching methods have been proposed to further enhance **DC** from the perspectives of data augmentation [48], feature alignment [44], and long-range trajectory matching [1].

However, these follow-up studies on gradient-matching methods fail to address a serious weakness that results from the discrepancy between training and testing phases. In the training phase, the trajectory of the weights generated by $\mathcal{S}$ is optimized to reproduce the trajectory of $\mathcal{T}$ which commenced from a set of weights that were progressively updated by $\mathcal{T}$. However, in the testing phase, the weights are no longer initialized by the weights with respect to $\mathcal{T}$, but the weights that are continually updated by $\mathcal{S}$ in previous iterations. The discrepancy of the starting points of the training and testing phases results in an error on the converged weights. Such inaccuracies will accumulate and have an adverse impact on the starting weight for subsequent iterations. As demonstrated in Figure 1, we observe the loss difference between the weights updated by $\mathcal{S}$ and $\mathcal{T}$. We refer to the error as the *accumulated trajectory error*, because it grows as the optimization algorithm progresses along its iterations.

The synthetic dataset $\mathcal{S}$ optimized by the gradient-matching methods is able to generalize to various starting weights, but is not sufficiently robust to mitigate the perturbation caused by the accumulated trajectory error of the

starting weights. To minimize this source of error, the most straightforward approach is to employ robust learning, which adds perturbations to the starting weights intentionally during training to make $\mathcal{S}$ robust to errors. However, such a robust learning procedure will increase the amount of information of the real dataset to be distilled. Given a fixed size of $\mathcal{S}$, the distillation from the increased information results in convergence issues and will degrade the final performance. We demonstrate this via empirical studies in subsection 3.2.

In this paper, we propose a novel approach to minimize the accumulated trajectory error that results in improved performance. Specifically, we regularize the training on the real dataset to a flat trajectory that is robust to the perturbation of the weights. Without increasing the information to be distilled in the real dataset, the synthetic dataset will enhance its robustness to the accumulated trajectory error at no cost. Thanks to the improved tolerance to the perturbation of the starting weights, the synthetic dataset is also able to ameliorate the accumulation of inaccuracies and improves the generalization during the testing phase. It can also be applied to cross-architecture scenarios. Our proposed method is compatible with the gradient-matching methods and boost their performances. Extensive experiments demonstrate that our solution minimizes the accumulated error and outperforms the vanilla trajectory matching method on various datasets, including CIFAR-10, CIFAR-100, subsets of the TinyImageNet, and ImageNet. For example, we achieve performance accuracies of 43.2% with only 10 images per class and 50.7% with 50 images per class on CIFAR-100, compared to the previous state-of-the-art work from [1] (which yields accuracies of only 40.1% and 47.7% respectively). In particular, we significantly improve the performance on a subset of the ImageNet dataset which contains higher resolution images by more than 4%.

2. Preliminaries and Related Work

Problem Statement. We start by briefly overviewing the problem statement of Dataset Distillation. We are given a real dataset $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{|\mathcal{T}|}$, where the examples $x_i \in \mathbb{R}^d$ and the class labels $y_i \in \mathcal{Y} = \{0, 1, \dots, C-1\}$ and C is the number of classes. *Dataset Distillation* refers to the problem of synthesizing a new dataset $\mathcal{S}$ whose size is much smaller than that of $\mathcal{T}$ (i.e., it contains much fewer pairs of synthetic examples and their class labels), such that a model f trained on the synthetic dataset $\mathcal{S}$ is able to achieve a comparable performance over the real data distribution $P_{\mathcal{D}}$ as the model f trained with the original dataset $\mathcal{T}$.

We denote the synthetic dataset $\mathcal{S}$ as $\{(s_i, y_i)\}_{i=1}^{|\mathcal{S}|}$ where $s_i \in \mathbb{R}^d$ and $y_i \in \mathcal{Y}$. Each class of $\mathcal{S}$ contains `ipc` (images per class) examples. In this case, $|\mathcal{S}| = \text{ipc} \times C$ and $|\mathcal{S}| \ll |\mathcal{T}|$. We denote the optimized weight parameters obtained by minimizing an empirical loss term over the

synthetic training set $\mathcal{S}$ as

$$\theta^S = \arg \min_{\theta} \sum_{(s_i, y_i) \in \mathcal{S}} \ell(f_{\theta}, s_i, y_i),$$

where ℓ can be an arbitrary loss function which is taken to be the cross entropy loss in this paper. *Dataset Distillation* aims at synthesizing a synthetic dataset $\mathcal{S}$ to be an approximate solution of the following optimization problem

$$\mathcal{S}_{\text{DD}} = \arg \min_{\mathcal{S} \subset \mathbb{R}^d \times \mathcal{Y}, |\mathcal{S}| = \text{ipc} \times C} L_{\mathcal{T}_{\text{Test}}}(f_{\theta^S}). \quad (1)$$

Wang et al. [45] proposed **DD** to solve $\mathcal{S}$ by optimizing Equation 1 after replacing $\mathcal{T}_{\text{Test}}$ with $\mathcal{T}$, i.e., minimizing $L_{\mathcal{T}}(f_{\theta^S})$ directly because $\mathcal{T}_{\text{Test}}$ is inaccessible.

Gradient-Matching Methods. Unfortunately, **DD**'s [45] performance is poor because optimizing Equation 1 only provides limited information for distilling the real dataset $\mathcal{T}$ into the synthetic dataset $\mathcal{S}$. This motivated Zhao et al. [50] to propose a so-called *gradient-matching method* **DC** to *match* the informative gradients calculated by $\mathcal{T}$ and $\mathcal{S}$ at each iteration to enhance the overall performance. Namely, they considered solving

$$\mathcal{S}_{\text{DC}} = \arg \min_{\substack{\mathcal{S} \subset \mathbb{R}^d \times \mathcal{Y} \\ |\mathcal{S}| = \text{ipc} \times C}} \mathbb{E}_{\theta_0 \sim P_{\theta_0}} \left[\sum_{m=1}^M \mathcal{L}(\mathcal{S}) \right], \quad \text{where} \quad (2)$$

$$\mathcal{L}(\mathcal{S}) = D(\nabla_{\theta_m} L_{\mathcal{S}}(f_{\theta_m}), \nabla_{\theta_m} L_{\mathcal{T}}(f_{\theta_m})). \quad (3)$$

In the definition of $\mathcal{L}(\mathcal{S})$, θ_m contains the weights updated from the initialization θ_0 with $\mathcal{T}$ at iteration m . The initial set of weights θ_0 is randomly sampled from an initialization distribution P_{θ} and M in Equation 2 is the total number of update steps. Finally, $D(\cdot, \cdot)$ in Equation 3 denotes a (cosine similarity-based) distance function measuring the discrepancy between two matrices and is defined as $D(X, Y) = \sum_{i=1}^I \left(1 - \frac{\langle X_i, Y_i \rangle}{\|X_i\| \|Y_i\|} \right)$, where $X, Y \in \mathbb{R}^{I \times J}$ and $X_i, Y_i \in \mathbb{R}^J$ are the i^{th} columns of X and Y respectively. At each distillation (training) iteration, **DC** [50] minimizes the $\mathcal{L}(\mathcal{S})$ as defined in Equation 3. The Gradient-Matching Method regularizes the distillation of $\mathcal{S}$ by matching the gradients of single-step (**DC** [50]), or multiple-step (**MTT** [1]) for improved performance. More related works can be found in Appendix B.

3. Methodology

The gradient-matching methods as discussed in section 2 constitute a reliable and state-of-the-art approach for dataset distillation. These methods match a short range of gradients with respect to a sets of the weights trained with the real dataset in the distillation (training) phase. However, the gradients calculated in the evaluation (testing) phase

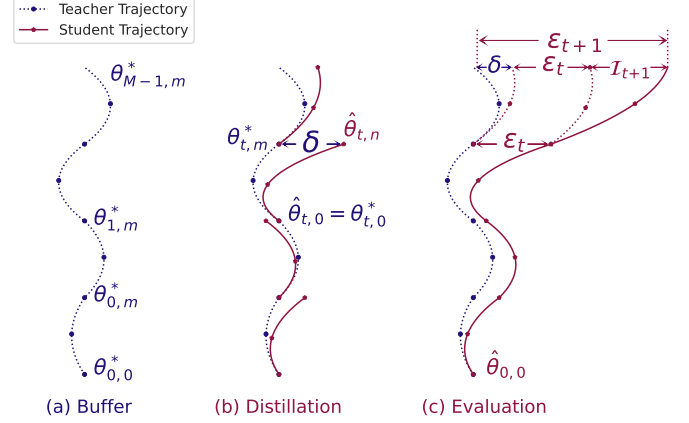


Figure 2. Illustration of trajectory matching: (a) A teacher trajectory is obtained by recording the intermediate network parameters at every epoch trained on the real dataset $\mathcal{T}$ in the buffer phase. (b) The synthetic dataset $\mathcal{S}$ is optimized to match the segments of the student trajectory with the teacher trajectory in the distillation phase. (c) The entire student trajectory and the accumulated trajectory error ϵ_t in the evaluation phase is shown. We aim to minimize this accumulated trajectory error.

are with respect to the recurrent weights from previous iterations, instead of the exact weights from the teacher's trajectory. Unfortunately, this discrepancy between the distillation (training) and evaluation (testing) phases result in a so-called *accumulated trajectory error*. We take **MTT** [1] as an instance of a gradient-matching method to explain the existence of such an error in subsection 3.2. We then propose a novel and effective method to mitigate the accumulated trajectory error in subsection 3.3.

3.1. Matching Training Trajectories (MTT)

In contrast to **DC** [50], **MTT** [1] matches the *accumulated gradients* over several steps (i.e., over a segment of the trajectory of updated weights), to further improve the overall performance. Therefore, **MTT** [1] solves for $\mathcal{S}$ as follows

$$\mathcal{S}_{\text{MTT}} = \arg \min_{\substack{\mathcal{S} \subset \mathbb{R}^d \times \mathcal{Y}, |\mathcal{S}| = \text{ipc} \times C \\ \theta_0 \sim P_{\theta_0}}} \mathbb{E} [\Delta \mathcal{A}], \quad \text{where} \quad (4)$$

$$\Delta \mathcal{A} = \|\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_0}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_0}), m]\|_2^2. \quad (5)$$

In Equation 5, the algorithm $\mathcal{A}$, which is the first-order optimizer sans momentum used in **MTT**, outputs the difference of the parameter vectors at the n^{th} iteration and at initialization, i.e.,

$$\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_0}), n] = \theta_n - \theta_0.$$

We model $\mathcal{A}$ as a function with input being the gradient $\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_0})$, which is run over a number of iterations n , and whose output is the accumulated change of weights after n iterations. Note that n, m are set so that $n < m$ because $|\mathcal{S}| \ll |\mathcal{T}|$. Equation 4 particularizes to Equation 2 when $n = m = 1$.

Intuitively, MTT [1] learns an informative synthetic dataset $\mathcal{S}$ so that it can provide sufficiently reliable information to the optimizer $\mathcal{A}$. Then, $\mathcal{A}$ utilizes the information from $\mathcal{S}$ to map the weights θ_0 sampled from its (initialization) distribution P_{θ_0} into an approximately-optimal parameter space $\mathcal{W} = \{\theta \mid L_{\mathcal{T}_{\text{test}}}(f_{\theta}) \leq L_{\text{tol}}\}$, where $L_{\text{tol}} > 0$ denotes an “tolerable minimum value”.

In the actual implementation, the ground truth trajectories, also known as the *teacher trajectories*, are prerecorded in the *buffer* phase as $(\theta_{0,0}^*, \dots, \theta_{0,m}^*, \theta_{1,0}^*, \dots, \theta_{M-1,m}^*)$. As illustrated in Figure 2(a), the teacher trajectories are trained until convergence on the real dataset $\mathcal{T}$ with a random initialization $\theta_{0,0}^*$. The long teacher trajectories are then partitioned into M segments $\{\Theta_t^*\}_{t=0}^{M-1}$ and each segment $\Theta_t^* = (\theta_{t,0}^*, \theta_{t,1}^*, \dots, \theta_{t,m}^*)$. Note that $\theta_{t,0}^* = \theta_{t-1,m}^*$ since the last set of weights of the segment will be used to initialize the first set of weights of the next one.

As shown in Figure 2(b), in the distillation phase, a segment of the weights Θ_t^* is randomly sampled from $\{\Theta_t^*\}_{t=0}^{M-1}$ and used to initialize the *student trajectory* $(\hat{\theta}_{t,0}, \hat{\theta}_{t,1}, \dots, \hat{\theta}_{t,n})$ which satisfies $\hat{\theta}_{t,0} = \theta_{t,0}^*$. In summary,

$$\begin{aligned}\theta_{t,m}^* &= \theta_{t,0}^* + \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t,0}^*}), m], \quad \text{and} \\ \hat{\theta}_{t,n} &= \hat{\theta}_{t,0} + \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\hat{\theta}_{t,0}}), n].\end{aligned}$$

Subsequently, MTT [1] solves Equation 4 by minimizing, at each distillation iteration, the following loss over $\mathcal{S}$:

$$\begin{aligned}\mathcal{L}(\mathcal{S}) &= \frac{\|\hat{\theta}_{t,n} - \theta_{t,m}^*\|_2^2}{\|\theta_{t,0}^* - \theta_{t,m}^*\|_2^2} \\ &= \frac{\|\theta_{t,0}^* + \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\hat{\theta}_{t,0}}), n] - \theta_{t,m}^*\|_2^2}{\|\theta_{t,0}^* - \theta_{t,m}^*\|_2^2} \\ &= \frac{\|\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\hat{\theta}_{t,0}}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t,0}^*}), m]\|_2^2}{\|\theta_{t,0}^* - \theta_{t,m}^*\|_2^2}.\end{aligned}$$

The synthetic dataset $\mathcal{S}$ is obtained by minimizing $\mathcal{L}(\mathcal{S})$ to be informative to guide the optimizer to update weights initialized at $\theta_{t,0}^*$ to eventually reach the target weights $\theta_{t,m}^*$.

3.2. Accumulated Trajectory Error

The student trajectory, to be matched in the distillation phase, is only one segment from $\hat{\theta}_{t,0}$ to $\hat{\theta}_{t,n}$ initialized from a precise $\theta_{t,0}^*$ from the teacher trajectory, i.e., $\hat{\theta}_{t,0} = \theta_{t,0}^*$. In the distillation phase, the *matching error* is defined as

$$\delta_t = \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t-1,m}^*}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t-1,m}^*}), m]. \quad (6)$$

and δ_t can be minimized in the distillation phase. However, in the actual evaluation phase, the optimization procedure of student trajectory is extended, and each segment is no longer initialized from the teacher trajectory but rather the last set

of weights in the previous segments, i.e., $\hat{\theta}_{t,0} = \hat{\theta}_{t-1,n}$. This discrepancy will result in a so-called *accumulated trajectory error*, which is the difference between the weights from the teacher and student trajectory in t^{th} segment, i.e.,

$$\epsilon_t = \hat{\theta}_{t+1,0} - \theta_{t+1,0}^* = \hat{\theta}_{t,n} - \theta_{t,m}^*$$

The initialization discrepancy between the distillation phase and the evaluation phase will incur an *initialization error* $\mathcal{I}_t = \mathcal{I}(\theta_{t,0}^*, \epsilon_t)$, representing the difference in accumulated gradients. It can be represented mathematically as:

$$\mathcal{I}_t = \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t,0}^* + \epsilon_t}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t,0}^*}), n], \quad (7)$$

In the next segment, ϵ_{t+1} can be derived as follows

$$\begin{aligned}\epsilon_{t+1} &= \hat{\theta}_{t+2,0} - \theta_{t+2,0}^* = \hat{\theta}_{t+1,n} - \theta_{t+1,m}^* \\ &= (\hat{\theta}_{t,n} + \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\hat{\theta}_{t,n}}), n]) \\ &\quad - (\theta_{t,m}^* + \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t,m}^*}), m]) \\ &= (\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\hat{\theta}_{t,n}}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t,m}^*}), m]) \\ &\quad + (\hat{\theta}_{t,n} - \theta_{t,m}^*) \\ &= (\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t,m}^* + \epsilon_t}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t,m}^*}), n]) \\ &\quad + (\mathcal{A}[\nabla_{\theta} L_{\mathcal{S}}(f_{\theta_{t,m}^*}), n] - \mathcal{A}[\nabla_{\theta} L_{\mathcal{T}}(f_{\theta_{t,m}^*}), m] + \epsilon_t) \\ &= \epsilon_t + \mathcal{I}(\theta_{t,m}^*, \epsilon_t) + \delta_{t+1}.\end{aligned} \quad (8)$$

The accumulated trajectory error ϵ_{t+1} continues to accumulate the initialization error $\mathcal{I}(\theta_{t,m}^*, \epsilon_t)$, the matching error ϵ_t , and the ϵ_t in previous segment. It also impacts the accumulation of errors in subsequent segments, and thereby degrading the final performance. This is illustrated in Figure 2(c). We conduct experiments to verify the existence of the accumulated trajectory error, which are demonstrated in Figure 1, more exploring about accumulated trajectory error can be found in Appendix A.1.

3.3. Flat Trajectory helps reduce the accumulated trajectory error

From Equation 8, we seek to minimize $\Delta\epsilon_{t+1} = \epsilon_{t+1} - \epsilon_t = \mathcal{I}_t + \delta_{t+1}$ where δ_{t+1} is the matching error of gradient-matching methods, which has been optimized to a small value in the distillation phase. However, the initialization error $\mathcal{I}_t$ is not optimized in the distillation phase. The existence of $\mathcal{I}_t$ results from the gap between the distillation and evaluation phases. To minimize it, a straightforward approach is to design the synthetic dataset $\mathcal{S}$ which is robust to the perturbation ϵ in the distillation phase. This is done by adding random noise to initialize the weights, i.e.,

$$\begin{aligned}\mathcal{S} &= \arg \min_{\substack{\mathcal{S} \subset \mathbb{R}^d \times \mathcal{Y}, \\ |\mathcal{S}| = \text{i.p.c} \times C}} \mathbb{E}_{\theta_0 \sim P_{\theta_0}, \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})} [\mathcal{L}(\mathcal{S}, \theta_0, \epsilon)], \quad \text{where} \\ \mathcal{L}(\mathcal{S}, \theta_0, \epsilon) &= \|\mathcal{A}[L_{\mathcal{S}}(f_{\theta_0 + \epsilon}), n] - \mathcal{A}[L_{\mathcal{T}}(f_{\theta_0}), m]\|_2^2, \quad (9)\end{aligned}$$

and $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ is a Gaussian with mean $\mathbf{0}$ and covariance $\sigma^2 \mathbf{I}$. However, we find that solving Equation 9 results in a degradation of the final performance when the number of images per class of $\mathcal{S}$ is not large (e.g., $\text{ipc} \in \{1, 10\}$). It only can improve the final performance when $\text{ipc} = 50$. These experimental results are reported in Table 1 and labelled as ‘‘Robust Learning’’. A plausible explanation is that adding random noise to the initialized weights $\theta_0 + \epsilon$ in the distillation phase is equivalent to mapping a more dispersed (spread out) distribution $P_{\theta_0 + \epsilon}$ into the parameter space $\mathcal{W} = \{\theta \mid L_{\mathcal{T}_{\text{Test}}}(f_\theta) \leq L_{\text{tol}}\}$, which necessitates more information per class (i.e., larger ipc) from $\mathcal{S}$ in order to ensure convergence, hence degrading the distilling effectiveness when $\text{ipc} \in \{1, 10\}$ is relatively small.

We thus propose an alternative approach to regularize the teacher trajectory to a *Flat Trajectory for Distillation* (FTD). Our goal is to distill a synthetic dataset whose standard training trajectory is flat; in other words, it is robust to the weight perturbations with the guidance of the teacher trajectory. Without exceeding the capacity of information per class (ipc), FTD improves the buffer phase to make the teacher trajectory robust to weight perturbation. As such, the flat teacher trajectory will guide the distillation gradient update to synthesize a dataset with the flat trajectory characteristic in a standard optimization procedure.

We aim to minimize $\mathcal{I}_t$ to ameliorate the adverse effect caused by ϵ_t . Assuming that $\|\epsilon_t\|_2^2$ is small, we can first rewrite the accumulated trajectory error Equation 8 using a first-order Taylor series approximation as $\mathcal{I}_t = \mathcal{I}(\theta_t^*, \epsilon_t) = \langle \frac{\partial \mathcal{A}}{\partial \epsilon_t}, \epsilon_t \rangle + O(\|\epsilon_t\|^2) \mathbf{1}$ (where $\mathbf{1}$ is the all-ones vector). To solve for θ_t^* that approximately minimizes the ℓ_2 norm of $\mathcal{I}(\theta_t^*, \epsilon_t)$ in the buffer phase, we note that

$$\begin{aligned} \theta_t^* &= \arg \min_{\theta_t} \|\mathcal{I}(\theta_t^*, \epsilon_t)\|_2^2 \approx \arg \min_{\theta_t} \left\| \frac{\partial \mathcal{A}}{\partial \epsilon_t} \right\|_2^2 \\ &= \arg \min_{\theta_t} \left\| \frac{\partial \mathcal{A}}{\partial \nabla_\theta L_{\mathcal{S}}(f_{\theta_t^*})} \cdot \frac{\partial \nabla_\theta L_{\mathcal{S}}(f_{\theta_t^*})}{\partial \theta} \cdot \frac{\partial \theta}{\partial \epsilon_t} \right\|_2^2. \end{aligned} \quad (10)$$

Since $\mathcal{A}$ is the first-order optimizer sans momentum, which has been modeled as a function as discussed after Equation 4. Therefore, $\frac{\partial \mathcal{A}}{\partial \nabla_\theta L_{\mathcal{S}}(f_{\theta_t^*})} = \eta$, where η is the learning rate used in $\mathcal{A}$. Because $\theta = \theta_t^* + \epsilon$, we have $\frac{\partial \theta}{\partial \epsilon} = 1$. Substituting these derivatives into Equation 10, we obtain

$$\begin{aligned} \arg \min_{\theta_t} \|\mathcal{I}(\theta_t^*, \epsilon_t)\|_2^2 &\approx \arg \min_{\theta_t} \left\| \frac{\partial \nabla_\theta L_{\mathcal{S}}(f_{\theta_t^*})}{\partial \theta} \right\|_2^2 \\ &= \arg \min_{\theta_t} \left\| \nabla_\theta^2 L_{\mathcal{S}}(f_{\theta_t^*}) \right\|_2^2. \end{aligned} \quad (11)$$

Minimizing $\|\nabla_\theta^2 L_{\mathcal{S}}(f_{\theta_t^*})\|_2^2$ is obviously equivalent to minimizing the largest eigenvalue of the Hessian $\nabla_\theta^2 L_{\mathcal{S}}(f_{\theta_t^*})$. Unfortunately, the computation of the largest eigenvalue is expensive. Fortunately, the largest eigenvalue of $\nabla_\theta^2 L_{\mathcal{S}}(f_{\theta_t^*})$

has also be regarded as the sharpness of the loss landscape, which has been well-studied by many works such as SAM [11] and GSAM [51]. In our work, we employ GSAM to help solve Equation 11 to find a teacher trajectory that is as flat as possible. The sharpness $S(\theta)$, can be quantified using

$$S(\theta) \triangleq \max_{\epsilon \in \Psi} [L_{\mathcal{T}}(f_{\theta+\epsilon}) - L_{\mathcal{T}}(f_\theta)] \quad (12)$$

where $\Psi = \{\epsilon : \|\epsilon\|_2 \leq \rho\}$ and $\rho > 0$ is a given constant that determines the permissible norm of ϵ . Then, θ^* is obtained in the buffer phase by solving a minimax problem as follows,

$$\theta^* = \arg \min_{\theta} \{L_{\mathcal{T}}(f_\theta) + \alpha S(\theta)\}, \quad (13)$$

where α is the coefficient that balances the robustness of θ^* to the perturbation. From the above derivation, we see that a different teacher trajectory is proposed. This trajectory is robust to the perturbation of the weights in the buffer phase so as to reduce the accumulated trajectory error in the evaluation phase. The details about our algorithm and the optimization of θ^* can be found in Appendix A.3.2.

4. Experiments

In this section, we verify the effectiveness of FTD through extensive experiments. We conduct experiments to compare FTD to state-of-the-art baseline methods evaluated on datasets with different resolutions. We emphasize the cross-architecture performance and generalization capabilities of the generated synthetic datasets. We also conduct extensive ablation studies to exhibit the enhanced performance and study the influence of hyperparameters. Finally, we apply our synthetic dataset to neural architecture search and demonstrate its reliability in performing this important task.

4.1. Experimental Setup

We follow up the conventional procedure used in the literature on dataset distillation. Every experiment involves two phases—distillation and evaluation. First, we synthesize a small synthetic set (e.g., 10 images per class) from a given large real training set. We investigate three settings $\text{ipc} = 1, 10, 50$, which means that the distilled set contains 1, 10 or 50 images per class respectively. Second, in the evaluation phase on the synthetic data, we utilize the learnt synthetic set to train randomly initialized neural networks and test their performance on the real test set. For each synthetic set, we use it to train five networks with random initializations and report the mean accuracy and its standard deviation for 1000 iterations with a standard training procedure.

Datasets. We evaluate our method on various resolution datasets. We consider the CIFAR10 and CIFAR100 [23] datasets which consist of tiny colored natural images with

Table 1. Comparison of the performances trained with ConvNet [12] to other distillation methods on the CIFAR [23] and Tiny ImageNet [25] datasets. We reproduce the results of MTT [1]. We cite the reported results of other baselines from Cazenavette et al. [1]. We only provide our reproduced results of DC and MTT on the Tiny ImageNet dataset as previous works did not report their results on this dataset.

ipc	CIFAR-10			CIFAR-100			Tiny ImageNet	
	1	10	50	1	10	50	1	10
real dataset	84.8±0.1			56.2±0.3			37.6±0.4	
DC [50]	28.3±0.5	44.9±0.5	53.9±0.5	12.8±0.3	25.2±0.3	-	-	-
DM [49]	26.0±0.8	48.9±0.6	63.0±0.4	11.4±0.3	29.7±0.3	43.6±0.4	3.9±0.2	12.9±0.4
DSA [48]	28.8±0.7	52.1±0.5	60.6±0.5	13.9±0.3	32.3±0.3	42.8±0.4	-	-
CAFE [44]	30.3±1.1	46.3±0.6	55.5±0.6	12.9±0.3	27.8±0.3	37.9±0.3	-	-
CAFE+DSA	31.6±0.8	50.9±0.5	62.3±0.4	14.0±0.3	31.5±0.2	42.9±0.2	-	-
PP [28]	46.4±0.6	65.5±0.3	71.9±0.2	24.6±0.1	43.1±0.3	48.4±0.3	-	-
MTT [1]	46.2±0.8	65.4±0.7	71.6±0.2	24.3±0.3	39.7±0.4	47.7±0.2	8.8±0.3	23.2±0.2
MTT+Robust Learning	45.8±0.7	63.2±0.7	72.7±0.2	24.1±0.3	39.4±0.4	47.9±0.2	-	-
FTD	46.8±0.3	66.6±0.3	73.8±0.2	25.2±0.2	43.4±0.3	50.7±0.3	10.4±0.3	24.5±0.2

Table 2. The performance comparison trained with ConvNet on the 128×128 resolution ImageNet subset. We only cite the results of MTT [1], which is the only and first distillation method among the baselines to apply their method on the high-resolution ImageNet subsets.

ipc	ImageNette		ImageWoof		ImageFruit		ImageMeow	
	1	10	1	10	1	10	1	10
Real dataset	87.4±1.0		67.0±1.3		63.9±2.0		66.7±1.1	
MTT	47.7±0.9	63.0±1.3	28.6±0.8	35.8±1.8	26.6±0.8	40.3±1.3	30.7±1.6	40.4±2.2
FTD	52.2±1.0	67.7±0.7	30.1±1.0	38.8±1.4	29.1±0.9	44.9±1.5	33.8±1.5	43.3±0.6

the resolution of 32×32 from 10 and 100 categories, respectively. We conduct experiments on the Tiny ImageNet [25] dataset with the resolution of 64×64 . We also evaluate our proposed FTD on the ImageNet subsets with the resolution of 128×128 . These subsets are selected 10 categories by Cazenavette et al. [1] from the ImageNet dataset [4].

Baselines and Models. We compare our method to a series of baselines including Dataset Condensation [50] (DC), Differentiable Siamese Augmentation [48] (DSA), and gradient-matching methods Distribution Matching [49] (DM), Aligning Features [44] (CAFE), Parameter Pruning [28] (PP), and trajectory matching method [1] (MTT).

Following the settings of Cazenavette et al. [1], we distill and evaluate the synthetic set corresponding to CIFAR-10 and CIFAR-100 using 3-layer convolutional networks (ConvNet-3) while we move up to a depth-4 ConvNet for the images with a higher resolution (64×64) for the Tiny ImageNet dataset and a depth-5 ConvNet for the ImageNet subsets (128×128). We evaluate the cross-architecture classification performance of distilled images on four standard deep network architectures: ConvNet (3-layer) [12], ResNet [15], VGG [40] and AlexNet [24].

Implementation Details. We use $\rho = 0.01$, $\alpha = 1$ as the default values while implementing FTD. The same suite of differentiable augmentations [48] has been implemented as

in previous studies [1, 49]. We use the Exponential Moving Average (EMA) [46] for faster convergence in the distillation phase for the synthetic image optimization procedure. The details of the hyperparameters used in buffer phase, distillation phase of each setting (real epochs per iteration, synthetic updates per iteration, image learning rate, etc.) are reported in Appendix A.3.3. Our experiments were run on two RTX3090 and four Tesla V100 GPUs.

4.2. Results

CIFAR and Tiny ImageNet. As demonstrated in Table 1, FTD surpasses all baselines among the CIFAR-10/100 and Tiny ImageNet dataset. In particular, our proposed FTD achieves significant improvement with $\text{ipc} = 10, 50$ on the CIFAR-10/100 datasets. For example, our method improves MTT [1] by 2.2% on the CIFAR-10 dataset with $\text{ipc} = 50$, and achieves 3.5% improvement on the CIFAR-100 dataset with $\text{ipc} = 10$. Besides, the results under “MTT+Robust learning” are obtained by using Equation 9 as the objective function of MTT during the distillation phase. “MTT+Robust learning” boosts the performance of MTT by 1.1% and 0.2% with $\text{ipc} = 50$ on the CIFAR-10/100 datasets, respectively; However, it will incur a performance degradation with $\text{ipc} = 1, 10$. We have introduced “MTT+Robust learning” in subsection 3.3.

We visualize part of the synthetic sets for $\text{ipc} = 1, 10$ of the CIFAR-100 and Tiny ImageNet datasets in Figure 3. Our images look easily identifiable and highly realistic, which are akin to combinations of semantic features. We provide more additional visualizations in Appendix A.3.5.

ImageNet Subsets. The ImageNet subsets are significantly more challenging than the CIFAR-10/100 [23] and Tiny ImageNet [25] datasets, because their resolutions are much higher. This characteristic of the images makes it difficult for the distillation procedure to converge. In addition, the majority of the existing dataset distillation methods may result an out-of-memory issue when distilling high-resolution data. The ImageNet subsets contains 10 categories selected from ImageNet-1k [4] following the setting of MTT [1], which is the first distillation method which is capable of distilling higher-resolution (128×128) images. These subsets include ImageNette (assorted objects), ImageWoof (dog breeds), ImageFruits (fruits), and ImageMeow (cats) in conjunction with a depth-5 ConvNet.

As shown in Table 2, FTD outperforms MTT in every subset with a significant improvement.¹ For example, we significantly improve the performance on the ImageNette subset when $\text{ipc} = 1, 10$ by more than 4.5%.

Cross-Architecture Generalization The ability to generalize well across different architectures of the synthetic dataset is crucial in the real application of dataset distillation. However, the existing dataset distillation methods suffer from a performance degradation when the synthetic dataset is trained by the network with a different architecture than the one used in distillation [1, 44].

Here, we study the cross-architecture performance of FTD, compare it with three baselines, and report the results in Table 3. We evaluate FTD on CIFAR-10 with $\text{ipc} = 50$. We use three more different neural network architectures for evaluation: ResNet [15], VGG [40] and AlexNet [24]. The synthetic dataset is distilled with ConvNet (3-layer) [12]. The results show that synthetic images learned with FTD perform and generalize well to different convolutional networks. The performances of synthetic data on architectures distinct from the one used to distill should be utilized to validate that the distillation method is able to identify essential features for learning, other than merely the matching of parameters.

4.3. Ablation and Parameter Studies

Exploring the Flat Trajectory Many studies [6, 17, 18, 21, 29] have revealed that DNNs with flat minima can generalize better than ones with sharp minima. Although FTD encourages dataset distillation to seek a flat trajectory which terminates a flat minimum, the progress along a flat teacher trajectory, which minimizes the accumulated trajectory error, contributes primarily to the performance gain

¹The results of ImageNet subsets are cited exactly from [1].

Table 3. Cross-Architecture Results trained with ConvNet on CIFAR-10 with $\text{ipc} = 50$. We reproduce the results of MTT, and cite the results of DC and CAFE reported in Wang et al. [44].

Method	Evaluation Model			
	ConvNet	ResNet18	VGG11	AlexNet
DC	53.9±0.5	20.8±1.0	38.8±1.1	28.7±0.7
CAFE	55.5±0.4	25.3±0.9	40.5±0.8	34.0±0.6
MTT	71.6±0.2	61.9±0.7	55.4±0.8	48.2±1.0
FTD	73.8±0.2	65.7±0.3	58.4±1.6	53.8±0.9

of FTD. To verify this, we design experiments to demonstrate that the attainment of a flat minimum does not enhance the accuracy of the synthetic dataset. We implement Sharpness-Aware Minimization (SAM) [11] to bias the training over the synthetic dataset obtained from MTT to converge at a flat minimum. We term this as “MTT + Flat Minimum” and compare the results to FTD. A set values of $\rho \in \{0.005, 0.01, 0.03, 0.05, 0.1\}$ is tested for a thorough comparison. We report the comparison in Figure 4. It can be seen that a flatter minimum does not help the synthetic dataset to generalize well. We provide more theoretical explanation about it in Appendix A.2. Therefore, FTD’s chief advantage lies in the suppression of the accumulated trajectory error to improve dataset distillation.

Effect of EMA. We implement the Exponential Moving Average (EMA) with $\beta = 0.999$ in the distillation phase of FTD for enhanced convergence. While EMA contributes to the improvement, it is not the primary driver. The results of our proposed approach with and without EMA are presented in Table 4. We observe that EMA enhances the evaluation accuracies. However, our proposed regularization in the buffer phase for a flatter teacher trajectory contributes most significantly to the performance improvement.

We have also conducted a parameter study on the coefficient ρ and observed that $\rho = 0.01$ is the optimal value for each dataset considered. See Appendix A.3.1.

Table 4. Ablation study of FTD. FTD without EMA still significantly surpasses MTT.

ipc	CIFAR-100		Tiny ImageNet	
	10	50	1	10
MTT	39.7±0.4	47.7±0.2	8.8±0.3	23.2±0.2
FTD (w.o. EMA)	43.4±0.3	49.8±0.3	9.8±0.2	24.1±0.3
FTD	43.2±0.3	50.7±0.3	10.0±0.2	24.5±0.2

4.4. Neural Architecture Search (NAS)

To better demonstrate the substantial practical benefits of our proposed method FTD, we evaluate our method in *neural architecture search* (NAS). NAS is one of the important down-stream task of dataset distillation. It aims to find the best network architecture for a given dataset among a variety of architecture candidates. Dataset distillation uses

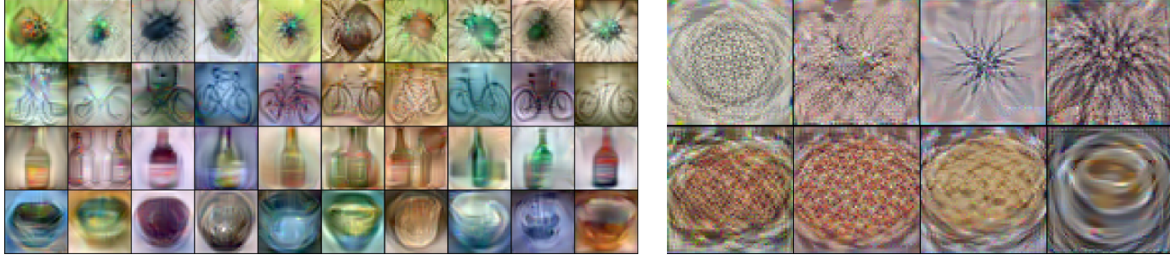


Figure 3. Visualization example of synthetic images distilled from 32×32 CIFAR-100 ($\text{ipc} = 10$), and 64×64 Tiny ImageNet ($\text{ipc} = 1$).

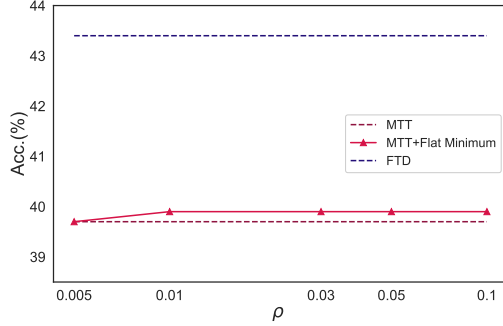


Figure 4. We apply SAM with different values of ρ on the synthetic dataset obtained from **MTT** to train the networks, which is termed as “MTT + Flat Minimum”. “MTT” and “FTD” represent the standard results of **MTT** and **FTD** on CIFAR-100 with $\text{ipc}=10$, respectively. A “flat” minimum does not help the synthetic dataset to generalize better.

the synthetic dataset as the proxy to efficiently search for the optimal architecture, which reduces the computational cost in a linear fashion. We show that **FTD** can synthesize a better and practical proxy dataset, which has a stronger correlation with the real dataset.

Following [50], we implement NAS on the CIFAR-10 dataset on a search space of 720 ConvNets that differ in network depth, width, activation, normalization, and pooling layers. More details can be found in Appendix A.3.4. We train these different architecture models on the **MTT** synthetic dataset, our synthetic dataset, and the real CIFAR-10 dataset for 200 epochs. Additionally, the accuracy on the test set of real data determines the overall architecture. The Spearman’s rank correlation between the searched rankings of the synthetic dataset and the real dataset training is used as the evaluation metric. Since the top-ranking architectures are more essential, only the rankings of the top 5, 10 and 20 architectures will be used for evaluation, respectively.

Our results are displayed in Table 5. **FTD** achieves much higher rank correlation than **MTT** in every top- k ranking. In particular, **FTD** achieves a 0.87 correlation in the top-5 ranking, which is very close to the value of 1.0 in real dataset, while **MTT**’s correlation is 0.41. **FTD** is thus able to obtain a reliable synthetic dataset, which generalizes well for NAS.

5. Conclusion and Future Work

We studied a flat trajectory distillation technique, that is able to effectively mitigate the adverse effect of the accumu-

Table 5. We implement NAS on CIFAR-10 with a search over 720 ConvNets. We present the Spearman’s rank correlation (1.00 is the best) of the top 5, 10, and 20 architectures between the rankings searched by the synthetic and real datasets. The Time column records the entire time to search for each dataset.

	Top 5	Top 10	Top 20	Time(min)	Images No.
Real	1.00	1.00	1.00	6,804	50,000
MTT	0.41	0.36	-0.04	360	500
FTD	0.87	0.68	0.54	360	500

lated trajectory error leading to significant performance gain. The cross-architecture and NAS experiments also confirmed **FTD**’s ability to generalize well across different architectures and downstream tasks of dataset distillation.

We note that the performance of the teacher trajectories in the existing gradient-matching methods doesn’t represent the state-of-the-art. This is because the optimization of the teacher trajectories has to be simplified to improve the convergence of distillation. The accumulation of the trajectory error, for instance, is a possible reason to limit the total number of training epochs of the teacher trajectories, that calls for further research.

Acknowledgements

This work is support by Joey Tianyi Zhou’s A*STAR SERC Central Research Fund (Use-inspired Basic Research) and the Singapore Government’s Research, Innovation and Enterprise 2020 Plan (Advanced Manufacturing and Engineering domain) under Grant A18A1b0045.

This work is also supported by 1) National Natural Science Foundation of China (Grant No. 62271432); 2) Guangdong Provincial Key Laboratory of Big Data Computing, The Chinese University of Hong Kong, Shenzhen (Grant No. B10120210117-KP02); 3) Human-Robot Collaborative AI for Advanced Manufacturing and Engineering (Grant No. A18A2b0046), Agency of Science, Technology and Research (A*STAR), Singapore; 4) Advanced Research and Technology Innovation Centre (ARTIC), the National University of Singapore (project number: A-0005947-21-00); and 5) the Singapore Ministry of Education (Tier 2 grant: A-8000423-00-00).

References

- [1] George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A Efros, and Jun-Yan Zhu. Dataset distillation by matching training trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4750–4759, 2022. 1, 2, 3, 4, 6, 7, 13
- [2] Jang Hyun Cho and Bharath Hariharan. On the efficacy of knowledge distillation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4794–4802, 2019. 1
- [3] Justin Cui, Ruochen Wang, Si Si, and Cho-Jui Hsieh. Dc-bench: Dataset condensation benchmark. *arXiv preprint arXiv:2207.09639*, 2022. 1
- [4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 1, 6, 7
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 1
- [6] Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. In *International Conference on Machine Learning*, pages 1019–1028. PMLR, 2017. 7
- [7] Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. In *International Conference on Machine Learning*, pages 1019–1028. PMLR, 2017. 13
- [8] Tian Dong, Bo Zhao, and Lingjuan Lyu. Privacy for free: How does dataset condensation help privacy? *arXiv preprint arXiv:2206.00240*, 2022. 2
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1
- [10] Jiawei Du, Hanshu Yan, Jiashi Feng, Joey Tianyi Zhou, Liangli Zhen, Rick Siow Mong Goh, and Vincent YF Tan. Efficient sharpness-aware minimization for improved training of neural networks. *arXiv preprint arXiv:2110.03141*, 2021. 13
- [11] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations*, 2020. 5, 7, 11, 12, 13
- [12] Spyros Gidaris and Nikos Komodakis. Dynamic few-shot visual learning without forgetting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4367–4375, 2018. 6, 7, 11
- [13] Jack Goetz and Ambuj Tewari. Federated learning via synthetic data. *arXiv preprint arXiv:2008.04489*, 2020. 2
- [14] Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, 2021. 1
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 6, 7
- [16] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, et al. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2(7), 2015. 1
- [17] Sepp Hochreiter and Jürgen Schmidhuber. Simplifying neural nets by discovering flat minima. In *Proceedings of the 8th International Conference on Neural Information Processing Systems*, pages 529–536, 1995. 7, 13
- [18] Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. In *International Conference on Learning Representations*, 2019. 7, 13
- [19] Haifeng Jin, Qingquan Song, and Xia Hu. Auto-keras: An efficient neural architecture search system. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 1946–1956, 2019. 2
- [20] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016. 13
- [21] Nitish Shirish Keskar, Jorge Nocedal, Ping Tak Peter Tang, Dheevatsa Mudigere, and Mikhail Smelyanskiy. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations*, 2017. 7
- [22] Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. *arXiv preprint arXiv:1606.07947*, 2016. 1
- [23] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. CIFAR-10 and CIFAR-100 datasets. URL: <https://www.cs.toronto.edu/kriz/cifar.html>, 6(1):1, 2009. 5, 6, 7
- [24] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. 6, 7
- [25] Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015. 6, 7
- [26] Shiye Lei and Dacheng Tao. A comprehensive survey to dataset distillation. *arXiv preprint arXiv:2301.05603*, 2023. 1
- [27] Guang Li, Ren Togo, Takahiro Ogawa, and Miki Haseyama. Soft-label anonymous gastric x-ray image distillation. In *2020 IEEE International Conference on Image Processing (ICIP)*, pages 305–309. IEEE, 2020. 2
- [28] Guang Li, Ren Togo, Takahiro Ogawa, and Miki Haseyama. Dataset distillation using parameter pruning. *arXiv preprint arXiv:2209.14609*, 2022. 6
- [29] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, 31, 2018. 7
- [30] Tengyuan Liang, Tomaso Poggio, Alexander Rakhlin, and James Stokes. Fisher–Rao metric, geometry, and complexity of neural networks. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 888–896. PMLR, 2019. 13

- [31] Chen Liu, Mathieu Salzmann, Tao Lin, Ryota Tomioka, and Sabine Süsstrunk. On the loss landscape of adversarial training: Identifying challenges and how to overcome them. *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 33:21476–21487, 2020. 13
- [32] Dougal Maclaurin, David Duvenaud, and Ryan Adams. Gradient-based hyperparameter optimization through reversible learning. In *International conference on machine learning*, pages 2113–2122. PMLR, 2015. 13
- [33] David A McAllester. Pac-bayesian model averaging. In *Proceedings of the twelfth annual conference on Computational learning theory*, pages 164–170, 1999. 11
- [34] Timothy Nguyen, Zhourong Chen, and Jaehoon Lee. Dataset meta-learning from kernel ridge-regression. *arXiv preprint arXiv:2011.00050*, 2020. 2, 13
- [35] Timothy Nguyen, Roman Novak, Lechao Xiao, and Jaehoon Lee. Dataset distillation with infinitely wide convolutional networks. *Advances in Neural Information Processing Systems*, 34:5186–5198, 2021. 1
- [36] Timothy Nguyen, Roman Novak, Lechao Xiao, and Jaehoon Lee. Dataset distillation with infinitely wide convolutional networks. *Advances in Neural Information Processing Systems*, 34:5186–5198, 2021. 1, 13
- [37] Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. Efficient neural architecture search via parameters sharing. In *International conference on machine learning*, pages 4095–4104. PMLR, 2018. 2
- [38] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4):1–34, 2021. 2
- [39] Andrea Rosasco, Antonio Carta, Andrea Cossu, Vincenzo Lomonaco, and Davide Bacciu. Distilled replay: Overcoming forgetting through synthetic samples. *arXiv preprint arXiv:2103.15851*, 2021. 2
- [40] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 6, 7
- [41] Felipe Petroski Such, Aditya Rawal, Joel Lehman, Kenneth Stanley, and Jeffrey Clune. Generative teaching networks: Accelerating neural architecture search by learning to generate synthetic training data. In *International Conference on Machine Learning*, pages 9206–9216. PMLR, 2020. 2
- [42] Ilia Sucholutsky and Matthias Schonlau. Soft-label dataset distillation and text dataset distillation. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021. 1
- [43] Paul Vicol, Jonathan P Lorraine, Fabian Pedregosa, David Duvenaud, and Roger B Grosse. On implicit bias in overparameterized bilevel optimization. In *International Conference on Machine Learning*, pages 22234–22259. PMLR, 2022. 1
- [44] Kai Wang, Bo Zhao, Xiangyu Peng, Zheng Zhu, Shuo Yang, Shuo Wang, Guan Huang, Hakan Bilen, Xinchao Wang, and Yang You. Cafe: Learning to condense dataset by aligning features. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12196–12205, 2022. 1, 2, 6, 7
- [45] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A Efros. Dataset distillation. *arXiv preprint arXiv:1811.10959*, 2018. 1, 2, 3, 13
- [46] Ross Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019. 6
- [47] Ruonan Yu, Songhua Liu, and Xinchao Wang. Dataset distillation: A comprehensive review. *arXiv preprint arXiv:2301.07014*, 2023. 1
- [48] Bo Zhao and Hakan Bilen. Dataset condensation with differentiable siamese augmentation. In *International Conference on Machine Learning*, pages 12674–12685. PMLR, 2021. 2, 6, 13
- [49] Bo Zhao and Hakan Bilen. Dataset condensation with distribution matching. *arXiv preprint arXiv:2110.04181*, 2021. 1, 2, 6
- [50] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. Dataset condensation with gradient matching. *ICLR*, 1(2):3, 2021. 1, 2, 3, 6, 8, 13
- [51] Juntang Zhuang, Boqing Gong, Liangzhe Yuan, Yin Cui, Hartwig Adam, Nicha Dvornek, Sekhar Tatikonda, James Duncan, and Ting Liu. Surrogate gap minimization improves sharpness-aware training. *arXiv preprint arXiv:2203.08065*, 2022. 5, 13